MagicCode Super-Resolution Software Known Issues

Document Version: v3.0.0. Release Date: October 24, 2026.

1. Scope

Known limitations for MagicCode Super-Resolution Software v3.0.0, including graphic super-resolution and Vulkan frame generation.

2. Functional constraints

- Input width/height must be 64 to 4032.
- Image scale [1, 8]; graphic (1, 8]. Not every scale is device-run.
- CPU backends have no graphic path.
- A default-created session (NULL handle on mc_nscaler_enable) cannot supply graphic extras. Graphic requires a ctrl_param_t create plus a temporal_frame_t every frame.
- Linux x86_64: lib/linux/libmagic_sr.a and libmagic_sr.so include CPU image (AVX2/FMA) plus Vulkan Graphic (Vulkan Temporal)/TAAU and FG (MAGIC_SR_HAS_FG). Rebuild with tools/build_linux_lib.sh; link -lm -lpthread. CPU graphic is unavailable.
- Derived reactive/transparency are approximations. Missing caller transparency is not a semantic mask; FSR-family paths bind zero transparency when omitted.
- FG is Vulkan only and requires MAGIC_SR_HAS_FG. Windows and Linux product builds define it. The Android Vulkan build currently does not; GEN create returns -100050 / -102013. D3D11 / Metal / GLES / OpenGL / CPU also return those codes.
- GEN_BALANCED needs a readable FG weight file via ctrl_param_t.model_path (or MAGIC_FG_WEIGHT_PATH / default search of magic_fg_core.bin). Recreate the handle to change weights. Missing or invalid weights: -100051 / -102014. FG pipeline create failure: -100052 / -102015.
- GEN interpolates only t=0.5. Color is output resolution; depth/MV are render resolution. Output is left in VK_IMAGE_LAYOUT_GENERAL. in_frame[0].width/height must match the handle render size or the handle is recreated.
- Non-GEN mc_nscaler_enable must still pass a two-pointer array; in_frame[1] is ignored and must not be read.
- Do not read mc_nscaler_version() / mc_build_id() after FreeLibrary/dlclose.
- Vulkan Speed (Lite-TAAU) now lets an explicit strong reactive mask suppress the still-camera accumulation branch. Previously that branch ignored the caller reactive mask when the camera was still or per-pixel MV was below about 0.2 px, so pixels with ~0 MV but changing content (TV / video-wall / LED-screen textures) could keep ghosting for about 64–96 frames after a content cut. Strong reactive (≥ about 0.85, fully effective at ≥ 0.95) now weakens that accumulation. Omitting reactive or passing a weak value (< 0.85) is unchanged.

Recommendation: mark screen-like objects whose MV is 0 but whose content changes (for example with CustomDepth stencil) as reactive at strength 1.0. Limit: Vulkan only in this release. The Metal / GLES / D3D11 Speed still-accumulation structure is the same idea but is not modified or verified here.

Graphic Super-Resolution

Graphic super-resolution runs through the nscaler session API: libmagic_sr.a (Windows: libmagic_sr.lib) + mc_interface.h + mc_nscaler_enable / mc_nscaler_control / mc_nscaler_disable. v2.2.0 replaced the v2.0.0 MC_Init / MC_Process / MC_Uninit entry and removed the v2.1.0 MC_Enable helper (mc_enable.h is deleted). A NULL *handle on mc_nscaler_enable creates a session with defaults (2x SR_SPEED_MODE_FOR_IMAGE) and processes the first frame. Pass mc_nscaler_input_frame_t *in2[2] = { &in, NULL } (set in2[1] only for GEN interpolate).

Modes

Value	Enumerator	Role
0	SR_SPEED_MODE_FOR_IMAGE	Image, throughput first
1	SR_BALANCED_MODE_FOR_IMAGE	Image, quality/cost trade-off
2	SR_SPEED_MODE_FOR_GRAPHIC	Graphic, throughput first
3	SR_BALANCED_MODE_FOR_GRAPHIC	Graphic, canonical FSR-family path
4	GEN_SPEED_MODE_FOR_GRAPHIC	FG-Fast (analytic, no weights). Vulkan + MAGIC_SR_HAS_FG. Combinable with SR bits; bare 4 equals SR_SPEED_MODE_FOR_IMAGE|GEN_SPEED.
8	GEN_BALANCED_MODE_FOR_GRAPHIC	FG-CoreV3. Same availability as Fast. Needs magic_fg_core.bin. Do not set both GEN bits.

- Graphic scaler_factor is (1, 8]; 1.0 itself is rejected. Image remains [1.0, 8.0].
- SR_SPEED_MODE_FOR_GRAPHIC on Metal, Vulkan, and OpenGLES currently uses a lightweight Convert → Activate → Upscale pipeline. Direct3D 11 and desktop OpenGL 4.3 Speed use the FSR-family graphic path (same family as Balanced, not a separate customer API).
- SR_BALANCED_MODE_FOR_GRAPHIC uses the canonical FSR-family path on every GPU graphic backend. Desktop Balanced edge-resolve is supported on Windows Vulkan, Windows Direct3D 11, desktop OpenGL 4.3, and Apple Metal. Android Vulkan and Android OpenGLES do not run edge-resolve. If enable_sharpening is on, RCAS runs after edge-resolve.
- CPU backends (X86 / NEON) cannot run graphic modes.
- The API accepts (1, 8]. Exact 2× has optimized kernels. Not every scale or device is claimed as device-run.
- Internal pipeline names are implementation detail, not public configuration.

Minimum per-frame inputs

- mc_nscaler_input_frame_t / mc_nscaler_output_frame_t: caller-owned color and output GPU resources.

- mc_nscaler_input_frame_t.frame: non-NULL temporal_frame_t* with struct_size = sizeof(temporal_frame_t).
- temporal_frame_t.depth and motion at input resolution.
- jitter_offset_x/y in input pixels, top-left, +X right, +Y down.
- frame_index / reset_history; camera_near / camera_far / camera_fov_y (radians).
- gpu_context at create (mc_nscaler_control SET_PARAM, or the first mc_nscaler_enable): Vulkan requires physical_device + device; D3D11 requires device; Metal device may be NULL; GL/GLES use the current context (library never makes current).
- Vulkan graphic requires mc_nscaler_input_frame_t.command_buffer to be a recording VkCommandBuffer. Metal command_buffer is optional. D3D11 / GL / GLES leave it NULL.
- Create-stage depth_reversed / depth_infinite / hdr_color on ctrl_param_t (all-zero = conventional-Z, finite far, LDR). Immutable for the handle lifetime.

Motion, depth, and masks

- Motion is current → previous. The library never negates MVs.
- motion_vector_scale_x/y is the only MV numeric conversion. (0,0) defaults to (input_width, input_height). Mixed zero is rejected.
- Depth is GPU device depth in [0, 1]. Linear view-space depth is not accepted.
- reactive / transparency are optional. Explicit caller textures win per channel.
- When omitted, the library derives an approximate mask. That is not a semantic transparency mask.
- Speed on Metal/Vulkan/GLES may derive both missing reactive and an approximate transparency. FSR-family paths (Balanced on all backends; Speed on D3D11/desktop GL) derive missing reactive only and bind an internal zero transparency texture.
- Both graphic modes enable depth+MV history rejection by default.
- Diagnostic only: MAGICSR_TAAU_AUTO_MASK=0 and MAGICSR_TAAU_HIST_REJECT=0 can turn those defaults off. They are not a customer configuration path.

3. Backend constraints

- Vulkan graphic: recording command buffer required; UNDEFINED layout rejected; no library-owned VkDevice.
- D3D11 graphic: gpu_context.device required. Enable_4params does not accept MAGIC_BACKEND_D3D11.
- GL/GLES: library never makes a context current.
- Linux Vulkan: same recording-command-buffer and layout contract as Windows Vulkan.
- Vendor/driver differences on Android Vulkan/GLES and Windows D3D11/Vulkan/GL remain.

4. Engine plugins

- Unity/UE public Enable APIs are image. Full graphic is Native session API.
- Unreal iOS packaging still depends on matching UE/Xcode/SDK versions.

5. Operational

- Ghosting if reset_history is not set on camera cuts or if MV/jitter contracts are wrong (current→previous, +Y down).
- frame_index != last+1 is a warning, not a hard fail; history may be stale.
- Do not use diagnostic environment kill switches as a product settings UI.

6. Mitigation

- Validate each target platform with production-like motion and depth.
- Use QUERY_STATUS and MC_ERROR_TEMPORAL_* codes.
- Keep Enable for single-texture image; keep session API for graphic.